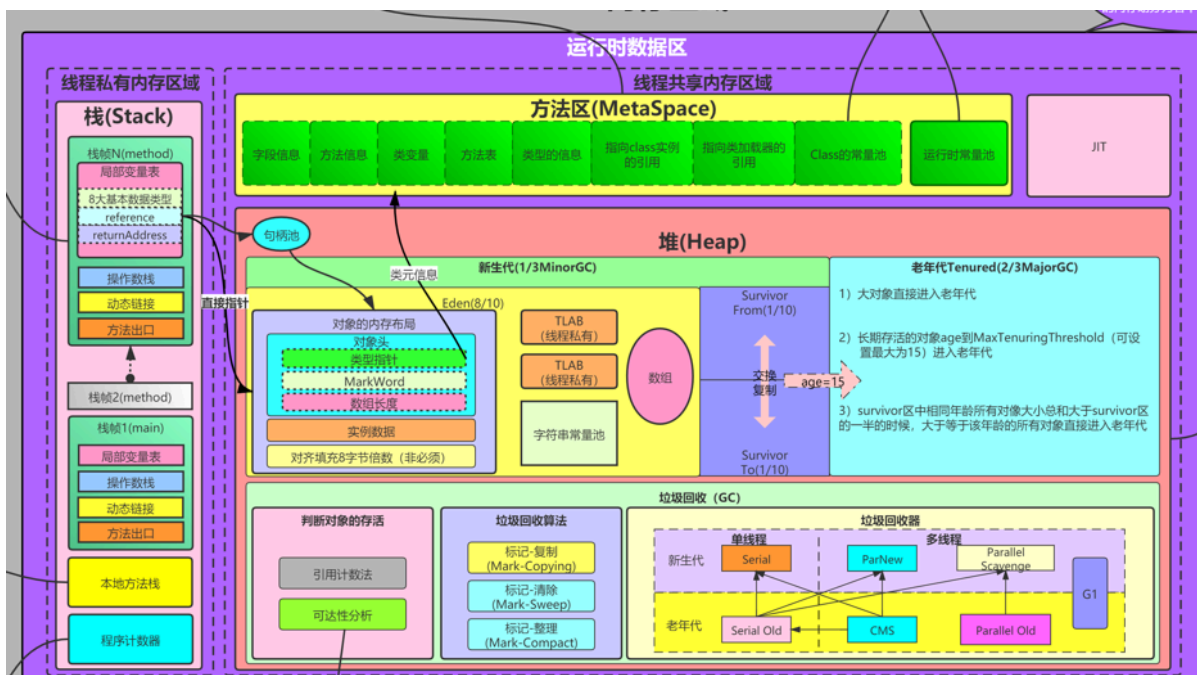


1、内存模型与垃圾回收（基础+进阶）

1、能介绍一下JVM的内存模型吗？

JVM内存主要分为线程私有和线程共享两大区域：

JVM 运行时数据区	
线程私有	线程共享
• 程序计数器 • 虚拟机栈 • 本地方法栈	• 堆 (Heap) - 新生代 (Eden + S0 + S1) - 老年代 • 方法区 (元空间) - 类信息、常量、静态变量 • 直接内存 (NIO Zero-Copy)



很多细节见：学生整理JVM技术图.png

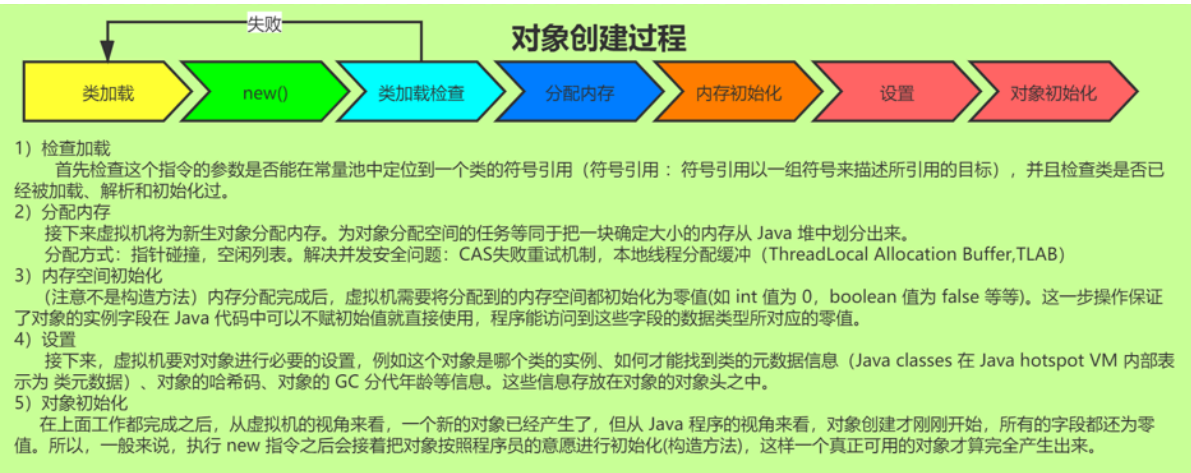
拓展点：实际工作中的问题：

- **OOM排查**：堆内存不足是最常见的线上问题
- **栈溢出**：递归调用过深导致StackOverflow
- **元空间溢出**：动态生成大量类（如Spring Boot热部署）

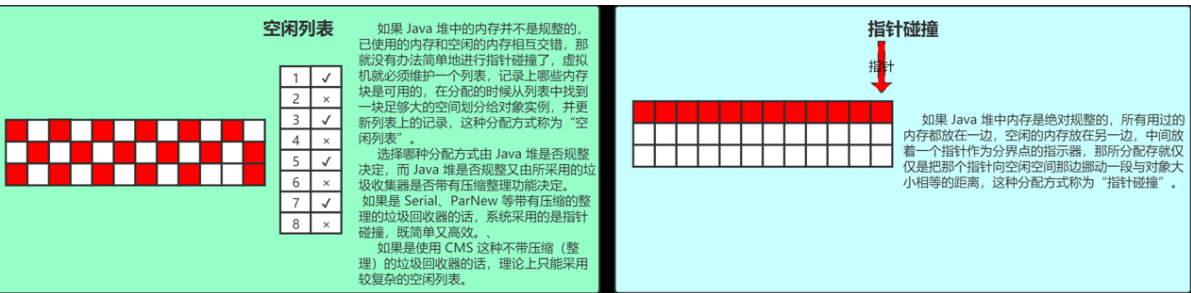
学员问题：AI时代的JDK会从21学起，JDK9有用吗？

以上内容，21也是这么玩。

2、对象创建过程中涉及哪些内存分配策略？

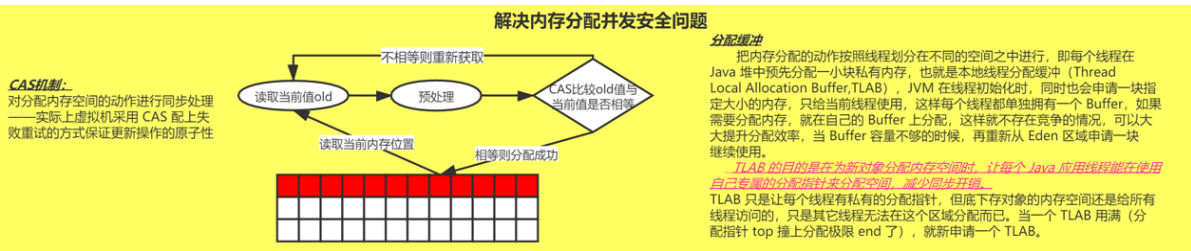


** 两种内存分配策略： **



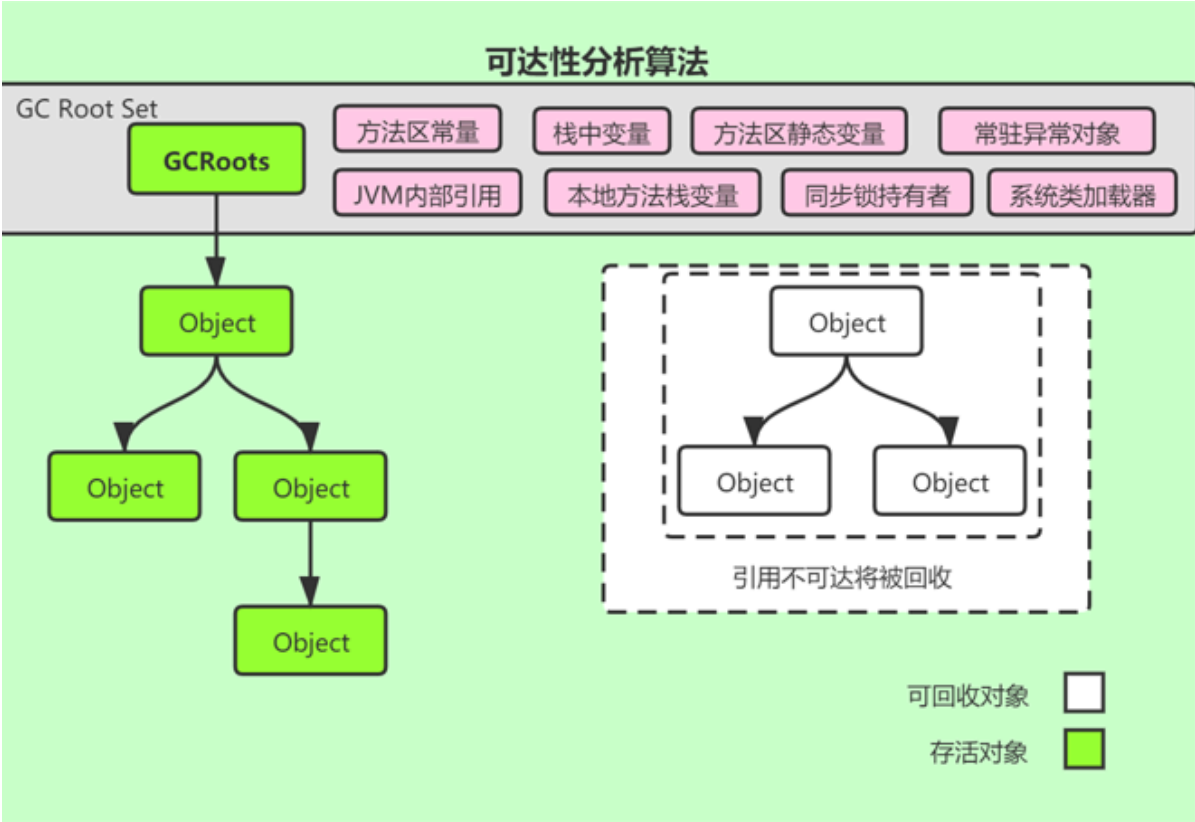
并发问题解决方案：

- CAS + 重试：乐观锁机制
- TLAB (Thread Local Allocation Buffer)：每个线程预先分配一小块内存，极大减少并发冲突



3、什么是可达性分析？哪些对象可以作为GC Roots？

可达性分析：从GC Roots向下搜索，搜索路径称为“引用链”。不在引用链上的对象可被回收。



拓展点：AI大模型服务（如基于Java的Transformer推理服务）可能长时间运行，内存泄漏会导致服务OOM。使用可达性分析可以定位泄漏的模型缓存或临时对象。

4、强引用、软引用、弱引用、虚引用有什么区别？

四种引用			
强引用 Object obj = new Object(), 就属于强引用。在任何情况下，只要有强引用关联（与根可达）还在，垃圾回收器就永远不会回收掉被引用的对象	软引用 SoftReference 系统将要发生内存溢出（OuyOfMemory）之前，这些对象就会被回收	弱引用 WeakReference 用弱引用关联的对象，只能生存到下一次垃圾回收之前，GC 发生时，不管内存够不够，都会被回收。	虚引用 PhantomReference 幽灵引用，最弱（随时会被回收掉）

演示代码如下：

软引用

```
package com.msb.jvm.ch2.reftype;

import java.lang.ref.SoftReference;
import java.util.LinkedList;
import java.util.List;

/**
 * 软引用
 * -Xms20m -Xmx20m
 */

public class TestSoftRef {
    //对象
    public static class User{
        public int id = 0;
        public String name = "";
        public User(int id, String name) {
            super();
            this.id = id;
            this.name = name;
        }
    }
}
```

```

    }
    @Override
    public String toString() {
        return "User [id=" + id + ", name=" + name + "]";
    }

}
//
public static void main(String[] args) {
    User u = new User(1,"lijin"); //new是强引用
    SoftReference<User> userSoft = new SoftReference<User>(u); //软引用
userSoft -> (软引用)    User对象
    u = null; //干掉强引用，确保这个实例只有userSoft的软引用
    //1、创建软引用后看存在不在
    System.out.println(userSoft.get()); //看一下这个对象是否还在
    //2、主动进行垃圾回收后看软引用后看存在不在
    System.gc(); //进行一次GC垃圾回收 千万不要写在业务代码中。
    System.out.println("After gc");
    System.out.println(userSoft.get());
    //3、主动导致OOM后看软引用后看存在不在
    //往堆中填充数据，导致OOM
    List<byte[]> list = new LinkedList<>();
    try {
        for(int i=0;i<100;i++) {
            //System.out.println("*****"+userSoft.get());
            list.add(new byte[1024*1024*1]); //1M的对象 100m
        }
    } catch (Throwable e) {
        //抛出了OOM异常时打印软引用对象
        System.out.println("Throwable*****"+userSoft.get());
        throw e;
    }

}
}
}

```

弱引用

```

package com.msb.jvm.ch2.reftype;

import java.lang.ref.WeakReference;

/**
 * 弱引用
 */
public class TestWeakRef {
    public static class User{
        public int id = 0;
        public String name = "";
        public User(int id, String name) {
            super();
            this.id = id;
            this.name = name;
        }
    }
}

```

```
@Override
public String toString() {
    return "User [id=" + id + ", name=" + name + "]";
}

}

public static void main(String[] args) {
    User u = new User(1,"king");
    WeakReference<User> userWeak = new WeakReference<User>(u);
    u = null;//干掉强引用，确保这个实例只有userWeak的弱引用
    System.out.println(userWeak.get());
    System.gc();//进行一次GC垃圾回收,千万不要写在业务代码中。
    System.out.println("After gc");
    System.out.println(userWeak.get());
}
}
```

拓展点：在大模型推理服务中，可以使用软引用**缓存加载好的模型（LLM模型体积大，内存不足时释放），**

避免重复加载。

5、垃圾回收算法有哪些？各有什么优缺点？

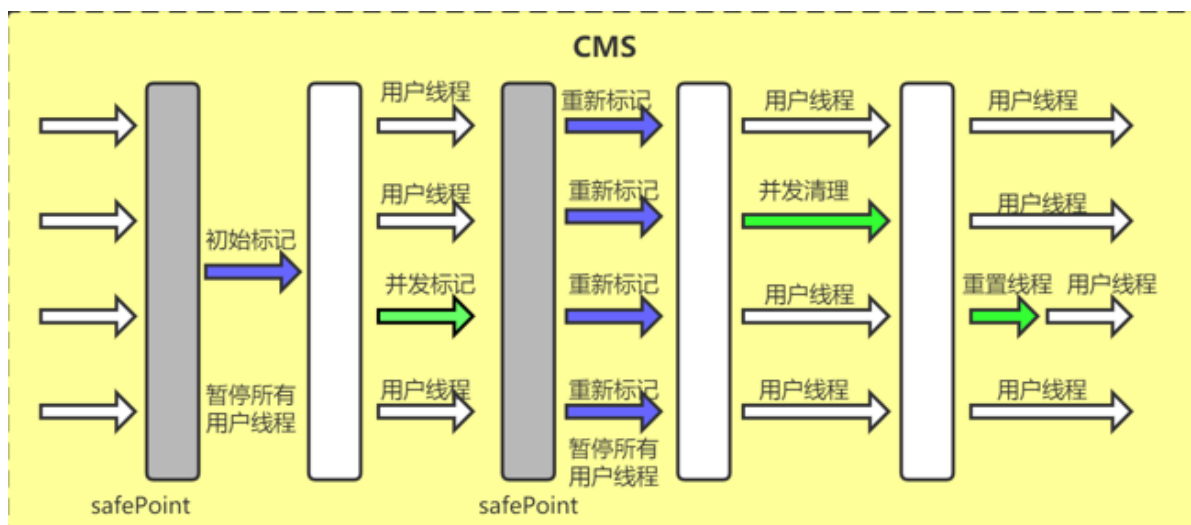
垃圾回收算法		
复制算法（Copying） 将可用内存按容量划分为大小相等的两块，每次只使用其中的一块。当这一块的内存用完了，就将还存活着的对象复制到另外一块上面，然后再把已使用过的内存空间一次清理掉。这样使得每次都是对整个半区进行内存回收，内存分配时就不用考虑内存碎片等复杂情况，只要按顺序分配内存即可，实现简单，运行高效。只是这种算法的代价是将内存缩小为了原来的一半。 但是要注意：内存移动是必须实打实的移动（复制），所以对应的引用(直接指针)需要调整。 复制回收算法适合于新生代，因为大部分对象朝生夕死，那么复制过去的对象比较少，效率自然就高。另外一半的一次性清理是很快。	标记-清除算法（Mark-Sweep） 分为“标记”和“清除”两个阶段：首先扫描所有对象标记出需要回收的对象，在标记完成后扫描回收所有被标记的对象，所以需要扫描两遍。回收效率略低，如果大部分对象是朝生夕死，那么回收效率降低，因为需要大量标记对象和回收对象，对比复制回收效率要低。 它的主要问题，标记清除之后会产生大量不连续的内存碎片，空间碎片太多可能会导致以后在程序运行过程中需要分配较大对象时，无法找到足够的连续内存而不需要提前触发另一次垃圾回收动作。 回收的时候如果需要回收的对象越多，需要做的标记和清除的工作越多， 所以标记清除算法适用于老年代。	标记-整理算法（Mark-Compact） 首先标记出所有需要回收的对象，在标记完成后，后续步骤不是直接对可回收对象进行清理，而是让所有存活的对象都向一端移动，然后直接清理掉端边界以外的内存。标记整理算法虽然没有内存碎片，但是效率偏低。 我们看到标记整理与标记清除算法的区别主要在于对象的移动，对象移动不单单会加重系统负担，同时需要全程暂停用户线程才能进行，届时所有引用对象的地方都需要更新（直接指针需要调整）。 所以看到，老年代采用的标记整理算法与标记清除算法，各有优点，各有缺点。

算法	原理	优点	缺点	适用区域
标记-清除	标记存活对象→清除	简单	内存碎片	老年代
复制	复制存活对象到Survivor	无碎片	浪费一半内存	新生代
标记-整理	标记→移动→更新引用	无碎片	移动成本高	老年代

分代收集理论：

- 新生代对象"朝生夕死"，用**复制算法**（Minor GC）
- 老年代对象存活久，用**标记-清除/整理**（Full GC）

6、能详细介绍一下CMS垃圾回收器吗？



1. 初始标记 (STW) → 标记GC Roots直接引用的对象
2. 并发标记 → 遍历引用链，标记所有存活对象
3. 重新标记 (STW) → 修正并发标记期间的变动
4. 并发清除 → 清除死亡对象

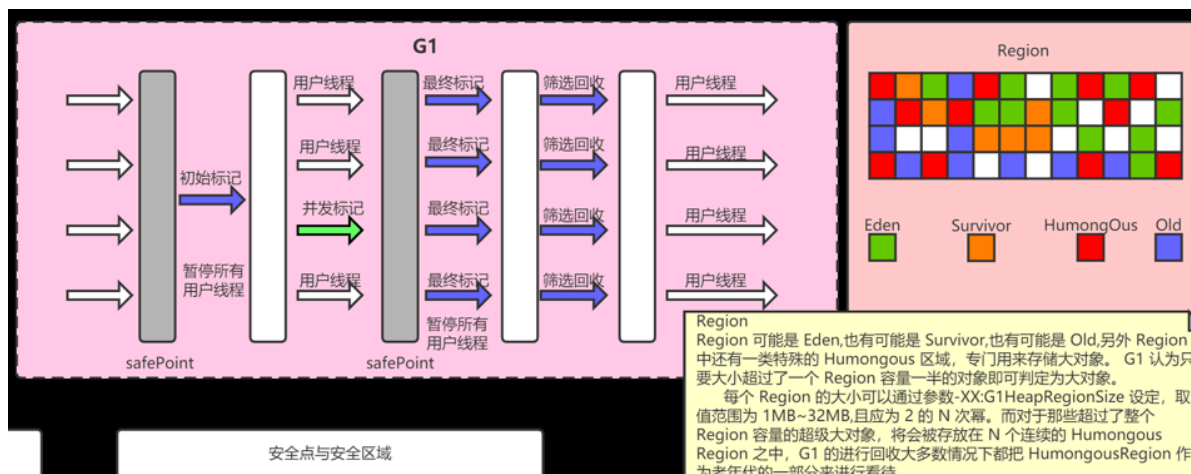
优点：并发标记，暂停时间短缺点：

- 产生内存碎片（标记-清除）
- 无法处理"浮动垃圾"
- 并发阶段占用CPU资源

拓展点：CMS已被deprecated** (JDK 14移除)，现在用G1或ZGC替代。

AI推理服务对延迟敏感，CMS的并发特性适合实时推理场景。但注意CMS无法处理大对象（因为老年代碎片化），对于大模型缓存场景可能不适用。

7、G1垃圾回收器了解吗？和CMS有什么区别？



核心概念：

- 将堆划分为多个大小相等的Region（1MB~32MB）
- 每个Region可以独立作为Eden或Survivor或Old区
- **Mixed GC**：同时回收年轻代和老年代

回收阶段：

1. 初始标记 (STW) → 标记Root Region
2. 并发标记 → 并发遍历所有Region
3. 最终标记 (STW) → 处理SATB日志
4. 筛选回收 (STW) → 根据回收价值选择Region

拓展点：大模型服务通常需要可预测的延迟，G1的Pause Time Target特性非常适合AI推理场景，可以避免因GC导致的服务抖动。

8、什么是STW (Stop-The-World) ？怎么优化？

STW：GC时必须暂停所有Java应用线程（除了GC线程）。

优化策略：

1. 选择合适的GC器
 - 低延迟需求 → G1 / ZGC / Shenandoah
 - 高吞吐需求 → Parallel GC
2. 减少GC频率
 - 增大堆内存
 - 优化对象分配（对象池化）
3. 减少STW时间
 - 调整GC参数
 - 使用并发GC（CMS、G1）
4. 提前发现风险

```
# 打印GC日志
-XX:+PrintGCDetails -Xloggc:gc.log

# 开启GC日志滚动
-XX:+UseGCLogFileRotation -XX:NumberOfGCLogFiles=5
```

```
package com.msb.jvm.ch1.oom;

import java.util.LinkedList;
import java.util.List;

/**
 * VM Args: -Xms30m -Xmx30m      堆的大小30M
 * 造成一个堆内存溢出(分析下JVM的分代收集)
 * GC调优---生产服务器推荐开启(默认是关闭的)
 * -XX:+HeapDumpOnOutOfMemoryError
 */
public class HeapOom2 {
    public static void main(String[] args) throws Exception {
        List<Object> list = new LinkedList<>();
        int i = 0;
        while(true){
```



```

        i++;
        if(i%1000==0) Thread.sleep(10);
        list.add(new Object());// 不能回收
    }

}
}

```

```

package com.msb.jvm.ch2;

import java.util.LinkedList;
import java.util.List;

/**
 * VM参数:
 * -Xms100M -Xmx100M -XX:+PrintGCDetails
 * -XX:+PrintGCDetails -XX:+UseSerialGC
 * -XX:+PrintGCDetails
 * -XX:+PrintGCDetails -XX:+UseConcMarkSweepGC
 * -XX:+PrintGCDetails -XX:+UseG1GC
 */
public class TestGC {

    /*不停往list中填充数据*/
    //就使用不断的填充 堆 -- 触发GC
    public static class FillListThread extends Thread{
        List<Object> list = new LinkedList<>();

        @Override
        public void run() {
            try {
                while(true){
                    if(list.size()*512/1024/1024>=512){
                        list.clear();
                        System.out.println("list is clear");
                    }
                    byte[] b1;
                    for(int i=0;i<100;i++){
                        b1 = new byte[512];
                        list.add(b1);
                    }
                    Thread.sleep(1);
                }

            } catch (Exception e) {
            }
        }
    }

    /*每100ms定时打印*/
    public static class TimerThread extends Thread{
        public final static long startTime = System.currentTimeMillis();
        @Override
        public void run() {

```



```
        try {
            while(true){
                long t = System.currentTimeMillis()-startTime;
                System.out.println(t/1000+"."+t%1000);
                Thread.sleep(100); //0.1s
            }

        } catch (Exception e) {
        }
    }
}

public static void main(String[] args) {
    //填充对象线程和打印线程同时启动
    FillListThread myThread = new FillListThread(); //造成GC, 造成STW
    TimerThread timerThread = new TimerThread(); //时间打印线程(每隔0.1s打印一次)
    myThread.start();
    timerThread.start();
}
}
```